

Exposition au monde

Route 53, CloudFront, ACM : un nom, un CDN, du HTTPS

CL-AWS1 — Jour 7 — De l'URL imprononçable au service présentable

Objectifs de la journée

À la fin de la journée, vous saurez :

- Gérer une **zone hébergée Route 53** : enregistrements, TTL, et surtout choisir la bonne **politique de routage** (simple, pondéré, latence, failover).
- Trancher **alias vs CNAME** sans hésiter (et savoir pourquoi l'apex n'accepte que l'alias).
- Monter une **distribution CloudFront** : origines S3 et ALB, cache, invalidation — et verrouiller un bucket privé avec **OAC**.
- Obtenir un **certificat ACM** (validation DNS), le placer au bon endroit (us-east-1 vs région !) et obtenir du **HTTPS de bout en bout**.
- Servir le **front StockLine** depuis un S3 privé derrière CloudFront.

Hier votre service répondait sur `abc-alb-1234.eu-west-3.elb.amazonaws.com` en HTTP. Personne ne met ça sur une carte de visite.

Plan de la journée

1. **Route 53** — zones, enregistrements, politiques de routage, health checks, alias vs CNAME.
2. **CloudFront** — distribution, origines, cache, invalidation, OAC pour S3 privé.
3. **ACM** — certificats publics gratuits, validation DNS, le piège us-east-1.
4. **Démo** — le front StockLine sur S3 privé derrière CloudFront, en HTTPS.

Particularité du jour : pas de domaine acheté (coût réel !) — on travaille avec le domaine `*.cloudfront.net` fourni, et on **explique précisément** ce qu'un vrai domaine changerait.

1. Route 53

Le DNS, en version AWS

Rappel express (CL-RÉSEAU J2) et présentation

Vous savez déjà : le DNS traduit des noms en adresses, par une hiérarchie de serveurs, avec des caches et des **TTL** partout. `dig` est votre ami.

Route 53 = le DNS managé d'AWS. Trois métiers dans un service :

1. **Enregistrer des domaines** (acheter `stockline.fr`, ~12 \$/an) ;

2. **Héberger des zones** : servir les enregistrements du domaine, avec un SLA de 100 % (le seul service AWS à l'afficher) ;
3. **Vérifier des santés** (health checks) et router intelligemment — le vrai plus par rapport à un DNS classique.

Le nom ? La convention : le DNS écoute sur le port **53**. Les « routes », c'est la suite de la matinée.

Zones hébergées

Une **zone hébergée** = le conteneur des enregistrements d'un domaine.

- **Zone publique** : répond au monde entier — notre sujet du jour. À la création, AWS assigne **4 serveurs de noms** (NS) ; si le domaine est acheté ailleurs (OVH, Gandi...), on reporte ces 4 NS chez le registrar : c'est la manœuvre de délégation.
- **Zone privée** : répond **uniquement dans vos VPC** (ex. `db.interne.local` → l'endpoint RDS). Très utilisé en entreprise.
- 💰 ****0,50 / mois par zone** + 0,40** par million de requêtes. C'est pour ça qu'on n'en crée pas une chacun aujourd'hui — mais tout se code pareil, avec ou sans.

```
# À titre de référence (nécessite un domaine réel) :  
aws route53 create-hosted-zone --name stockline.fr \  
--caller-reference $(date +%s)
```

Les enregistrements : le vocabulaire minimal

Déjà croisés avec **dig** en CL-RÉSEAU — dans une zone Route 53 :

Type	Rôle	Exemple
A / AAAA	nom → IPv4 / IPv6	<code>www</code> → <code>203.0.113.10</code>
CNAME	nom → autre nom	<code>blog</code> → <code>monblog.plateforme.io</code>
ALIAS	nom → ressource AWS (extension Route 53)	<code>www</code> → ALB, CloudFront, S3
MX	serveurs de mail	—
TXT	vérifications, anti-spam	validation ACM, SPF
NS / SOA	squelette de la zone (ne pas toucher)	—

Chaque enregistrement porte un **TTL** : durée de cache chez les résolveurs. TTL long = économe mais lent à changer ; TTL court = réactif mais plus de requêtes. Retenez : **avant une migration, on baisse le TTL à l'avance**.

Alias vs CNAME : le duel à connaître par cœur

	CNAME	ALIAS (Route 53)
Nature	Standard DNS universel	Extension propriétaire AWS
À l'apex (stockline.fr)	 Interdit (règle DNS : l'apex porte SOA+NS, le CNAME exclut tout autre enregistrement)	 Autorisé
Cible	N'importe quel nom	Ressources AWS (ALB, CloudFront, S3 site, ou un autre enregistrement de la zone)
Facturation des requêtes	Payante	Gratuite
Suit les IP changeantes du service	Oui (indirection)	Oui, résolu côté serveur (plus rapide)
Health check intégré	Non	Oui (« evaluate target health »)

La règle de décision, à réciter : **cible AWS** → **alias, toujours**. **Cible externe** → **CNAME, jamais à l'apex**.

Cas concret : [stockline.fr](#) → ALB : alias obligatoire (apex + cible AWS). [blog.stockline.fr](#) → plateforme externe : CNAME.

Politiques de routage (1/2) : simple et pondéré

Route 53 ne sait pas seulement répondre — il sait **choisir** sa réponse. Politique n°1 :

- **Simple** : une question, une réponse fixe. Le DNS de tout le monde. Suffit pour 90 % des enregistrements.
- **Pondéré (weighted)** : plusieurs enregistrements pour le même nom, chacun avec un **poids**. Route 53 répartit les réponses au prorata.
 - Cas d'usage roi : le **déploiement canary** — v2 à poids 5, v1 à poids 95, on observe, puis on glisse 20/80, 50/50... 100/0.
 - Réversible en **une modification d'enregistrement** : le rollback le plus rapide du métier.
 - Poids 0 = sortie propre du trafic.

Politiques de routage (2/2) : latence et failover

- **Latence (latency-based)** : plusieurs enregistrements, chacun étiqueté d'une **région AWS** ; Route 53 répond avec la cible **la plus rapide** pour le client (mesures réelles de latence, pas la géographie). Le réflexe multi-régions : Paris pour l'Europe, Montréal pour l'Amérique.
- **Failover** : deux enregistrements — **PRIMARY** (servi tant que son health check est vert) et **SECONDARY** (servi sinon). C'est le plan B automatisé : site principal → page de secours S3/CloudFront.
 -  Sans health check sur le primaire, le failover **n'existe pas**.

- À connaître de nom : **géolocalisation** (« les clients DE Suède », logique réglementaire/langue — différent de la latence !), géoproximité, multivalue.

Et surtout : **les politiques s'imbriquent** — un failover dont le primaire est un groupe latence, dont la branche Europe est un pondéré... C'est l'exercice 7-1.

Health checks Route 53

Le juge de paix du failover : ~15 sondes mondiales interrogent votre cible (toutes les 30 s par défaut, 10 s en rapide).

- Modes : requête **HTTP(S)** (code 2xx/3xx attendu), **TCP**, recherche de **chaîne** dans la réponse, ou état d'une **alarme CloudWatch** (pont avec le J8 !).
- Un enregistrement failover PRIMARY **référence** un health check ; les alias peuvent activer *evaluate target health* (l'avis des health checks de l'ALB, gratuit).
- 💰 ~0,50 \$/mois par health check sur cible AWS.

⚠ Le piège vu à l'exercice 7-1 : un health check HTTP ne lit que le **code de statut**. Si `/sante` renvoie 200 avec `"base_de_donnees": "indisponible"` dans le corps... le DNS croit que tout va bien. **Une application doit dire sa vérité dans son code de statut** (503 quand la base est morte).



Exercice 7-1 — Architecte DNS : le plan de routage de NordikShop

Fichier : `exercices/06-cl-aws1/exercice-7-1-plan-de-routage-route53.md` — **40 min, en binômes**

Un e-commerçant, deux ALB (Paris, Montréal), une page de secours CloudFront, une v2 à tester sur 5 % du trafic, un blog externe :

- concevoir les **5 enregistrements** (nom, type, politique, health check, justification) ;
- répondre aux questions qui fâchent : pourquoi pas de CNAME à l'apex, le health check qui croit un menteur, le TTL pendant un failover.

Sur papier — un vrai domaine coûte de l'argent réel. C'est LE format de question d'architecture de la SAA : autant s'y entraîner dès maintenant.

2. CloudFront

Le CDN : rapprocher le contenu des utilisateurs

Le problème que CloudFront résout

Votre front est dans un bucket S3 à Paris. Trois problèmes :

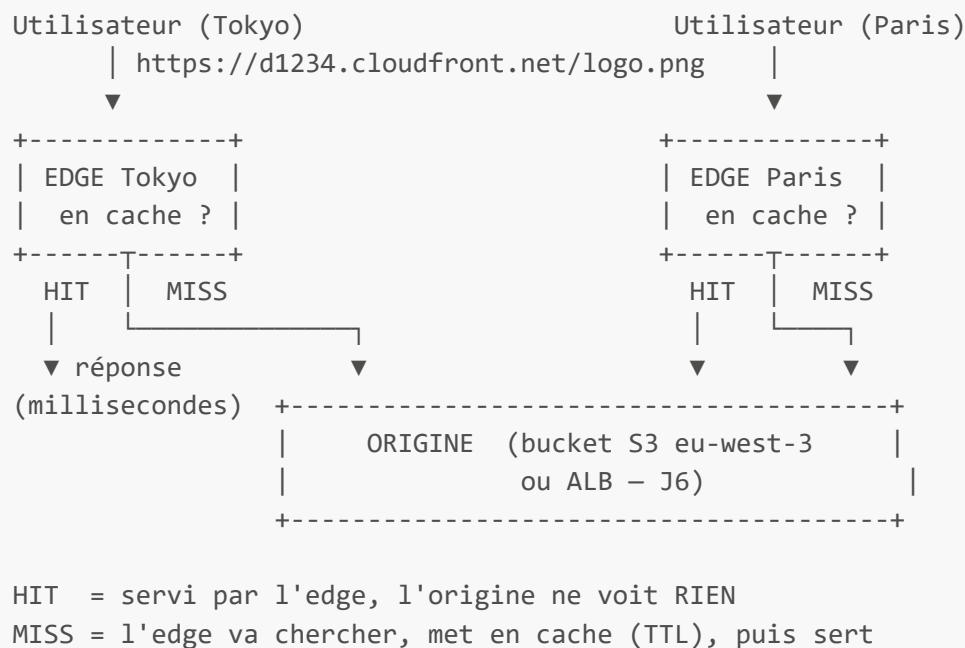
1. **La distance** : un utilisateur à Tokyo fait l'aller-retour Paris à chaque fichier — 250 ms de latence incompressible, sur chaque image.

2. **La charge à l'origine** : 100 000 visiteurs = 100 000 fois les mêmes GET sur le bucket (ou pire, sur votre ALB).
3. **Le HTTP du J4** : l'hébergement statique S3 ne fait pas de HTTPS avec un nom propre — indéfendable en 2026.

Un **CDN (Content Delivery Network)** répond aux trois : des centaines de **points de présence (edges)** dans le monde gardent une **copie en cache** du contenu, près des utilisateurs — et parlent HTTPS nativement.

CloudFront = le CDN d'AWS : 600+ edges, intégré à S3, ALB, ACM, Route 53.

La distribution : le trajet d'une requête



Origines et comportements : une distribution, plusieurs mondes

Une distribution CloudFront assemble :

- des **origines** : d'où vient le contenu — un bucket **S3** (notre front), un **ALB** (notre API), n'importe quel serveur HTTP ;
- des **comportements (behaviors)** : quel chemin va vers quelle origine, avec quelle politique de cache. Le premier motif qui matche gagne.

```

https://app.stockline.fr
  /api/*      —> origine ALB  politique CachingDisabled
                  (du dynamique : on ne cache PAS)
  /* (défaut) —> origine S3   politique CachingOptimized
                  (du statique : on cache à fond)
  
```

C'est le montage exact du TP2 : **un seul domaine**, le front ET l'API, chacun son régime de cache. Bonus : plus de problème CORS (même origine pour le navigateur).

Le cache : TTL et politiques

Chaque objet en cache a une durée de vie (**TTL**), déterminée par la **politique de cache** du comportement et les en-têtes de l'origine (**Cache-Control**) :

- Politiques **managées** (on les référence, on ne réinvente pas) :
 - **CachingOptimized** — TTL par défaut 24 h : le statique ;
 - **CachingDisabled** — rien n'est mis en cache : le dynamique, les API.
- L'origine peut préciser objet par objet : `aws s3 cp --cache-control "max-age=60"` — la donnée déclare sa propre fraîcheur.
- Diagnostic en une commande : `curl -sI ... | grep x-cache` → `Hit from cloudfront / Miss from cloudfront`.

À bien comprendre : le cache est **par edge**. Paris peut servir la v1 pendant que Tokyo (cache vierge) sert la v2. « Le CDN a la dernière version » est une phrase qui n'a pas de sens.

Mettre à jour : invalidation vs versionnement

Vous poussez `index.html` v2 dans le bucket... et **rien ne change** : les edges servent leur copie jusqu'à expiration du TTL (24 h !). Deux issues :

- **L'invalidation** — l'outil d'urgence :

```
aws cloudfront create-invalidation \  
  --distribution-id E2EXAMPLE123 --paths "/index.html"
```

Effet en ~1 min sur tous les edges. 💰 1 000 chemins gratuits/mois, puis 0,005 \$ pièce. Invalider `/*` à chaque déploiement = taux de cache ruiné + facture.

- **Le versionnement des noms** — l'outil de déploiement : `app.3f2a1.js` (hash dans le nom). Contenu nouveau = URL nouvelle = **jamais besoin d'invalidier**. Seul `index.html` garde un TTL court. La norme en production.

OAC : le bucket redevient privé

AVANT (J4)
bucket PUBLIC en HTTP

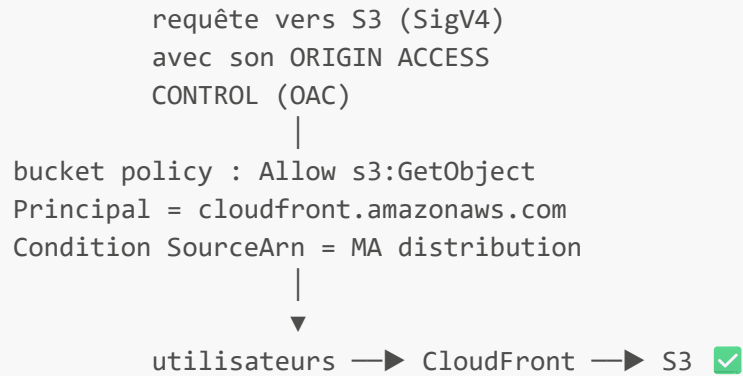
n'importe qui → S3 🤖

AUJOURD'HUI (OAC)
bucket PRIVÉ (Block Public Access ON)

n'importe qui → S3 : 403 ✅



CloudFront signe chaque



La bucket policy OAC — relue avec la grille du J4

```

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "cloudfront.amazonaws.com" },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::PREFIX-stockline-front/*",
    "Condition": { "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::COMPTE:distribution/ID_DISTRIB" } }
  }]
}
  
```

- **Principal** : le service CloudFront — pas un utilisateur, pas *.
- **Resource** : les **objets** (/*) — `s3:GetObject` ne porte pas sur le bucket (l'oubli du /* = LA panne classique).
- **Condition** : **votre** distribution, identifiée par ARN — même un autre client CloudFront est refusé.

(L'ancêtre **OAI** traîne dans les vieilles docs : en 2026, c'est **OAC**.)

💰 CloudFront : la bonne surprise du bloc

💰 Encadré coût — CloudFront

- Free-tier **permanent** : 1 To de sortie + 10 M de requêtes **par mois** ;
- une distribution **à trafic nul ne coûte rien** (contrairement à l'ALB !) ;
- invalidations : 1 000 chemins/mois gratuits ;
- `PriceClass_100` (edges NA + Europe) : suffisant et moins cher que le monde entier.

Nos démos et le TP2 tiennent **très largement** dans le gratuit. On détruit quand même les distributions en fin de journée : un compte propre est un compte auditable — et une distribution oubliée devant un bucket oublié, c'est une porte d'entrée oubliée.

Attention quand même en entreprise : à fort trafic, le **data transfer sortant** est LE poste de coût des CDN.

3. ACM — AWS Certificate Manager

Le HTTPS sans acheter de certificat ni gérer d'expiration

ACM : ce que c'est, ce que ça change

Rappel CL-RÉSEAU J3 : HTTPS = TLS = un **certificat** prouvant que vous êtes bien **stockline.fr**, signé par une autorité, à renouveler avant expiration. Historiquement : payant, et une astreinte à renouvellement (qui n'a jamais vu un site tomber pour un certificat expiré ?).

ACM change les deux termes :

- certificats publics **gratuits**, illimités ;
- **renouvellement automatique** tant que la validation DNS reste en place ;
- la clé privée ne sort **jamais** d'AWS : le certificat s'**attache** aux services intégrés (ALB, CloudFront, API Gateway) — il ne se télécharge pas.

Conséquence : ACM ne fournit pas de certificat à installer sur une EC2. Pour le maillon ALB → instance, on verra autre chose (dans deux slides).

Demander et valider un certificat

```
aws acm request-certificate \  
  --domain-name stockline.fr \  
  --subject-alternative-names "*.stockline.fr" \  
  --validation-method DNS \  
  --region us-east-1          # pour CloudFront ! (slide suivante)
```

La **validation DNS** — prouvez que le domaine est à vous :

1. ACM génère un enregistrement **CNAME** au nom/valeur aléatoires ;
2. vous le créez dans la zone Route 53 (un bouton si la zone est chez AWS) ;
3. ACM le détecte → certificat **émis** (minutes) ;
4. tant que le CNAME reste en place → **renouvellements automatiques à vie**.

C'est pour le point 4 qu'on choisit DNS plutôt que la validation e-mail (manuelle à chaque renouvellement). Le wildcard ***.stockline.fr** couvre tous les sous-domaines de premier niveau.

LE piège : la région du certificat

Un certificat ACM vit **dans une région**, et le service qui le consomme impose laquelle :

Le certificat est pour...	Région ACM obligatoire
CloudFront	us-east-1, toujours (service global, plan de contrôle en Virginie)

Le certificat est pour... Région ACM obligatoire

ALB	la région de l'ALB (eu-west-3 pour nous)
API Gateway	selon le type — retenez la logique, pas la liste

Donc pour `app.stockline.fr` → CloudFront → ALB en HTTPS : **deux certificats** (un en us-east-1 pour la distribution, un en eu-west-3 pour le listener HTTPS de l'ALB).

Symptôme du piège : « mon certificat n'apparaît pas dans la liste déroulante de CloudFront » → il est dans la mauvaise région. Question d'examen quasi garantie, incident de production très réel.

HTTPS de bout en bout : les maillons

```

Navigateur —TLS #1—► CloudFront —TLS #2—► ALB —(#3)—► EC2
      cert ACM                cert ACM                HTTP en clair
      us-east-1              eu-west-3                DANS le VPC
      (app.stockline.fr)    (listener 443)          (SG verrouillés)
  
```

```

#1 ViewerProtocolPolicy: redirect-to-https → jamais de HTTP client
#2 OriginProtocolPolicy: https-only       → CloudFront exige TLS
#3 choix assumé du TP2 : réseau privé + SG font le travail
    (sinon : certificat auto-signé sur l'instance – l'ALB ne
      vérifie pas l'autorité, il veut juste du TLS)
  
```

Sans domaine acheté : TLS #1 existe quand même, gratuitement, via le certificat `*.cloudfront.net` d'AWS — c'est notre démo.



Exercice 7-2 — CloudFront sous le capot : cache, OAC et certificats

Fichier : `exercices/06-cl-aws1/exercice-7-2-cloudfront-cache-et-oac.md` — **35 min, seul(e)**

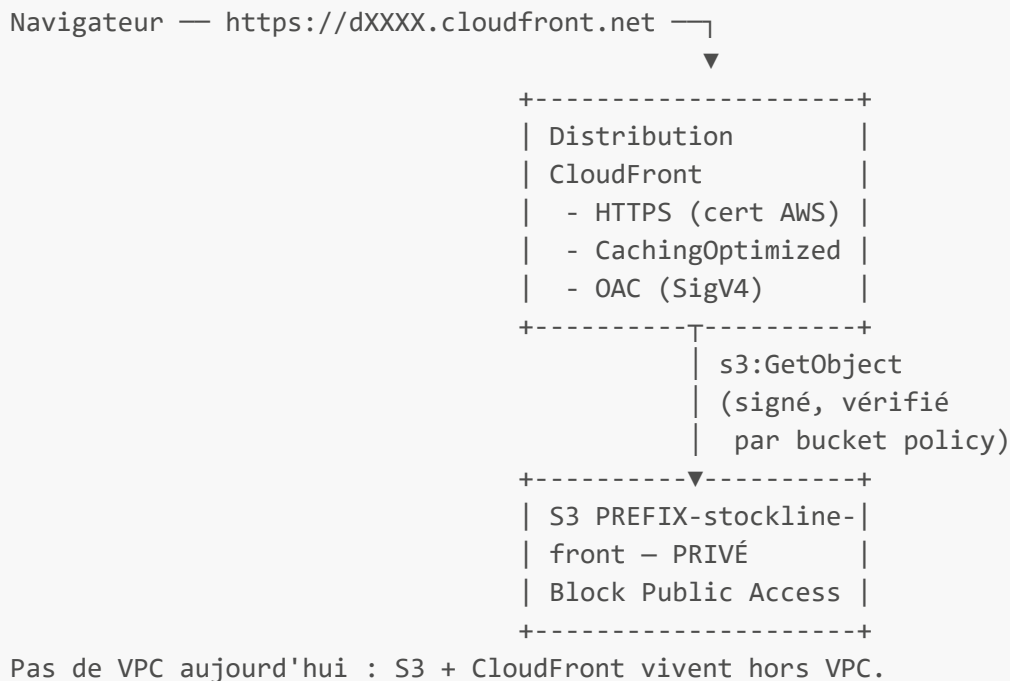
Audit d'une mise en production :

- **prédire le cache** : qui voit la v1, qui voit la v2, ce que fait l'invalidation, le cas du `stocks.json` périmé, le procès du `/*` quotidien ;
- **debugger une bucket policy OAC** qui renvoie 403 (deux erreurs exactement) ;
- **placer les certificats** pour `app.stockline.fr` → CloudFront → ALB (combien, où, validés comment).

Tout sur papier. Le raisonnement « par edge et par TTL » que vous y construisez est celui qui évite les « mais j'ai poussé le fichier ! » du TP2.

4. Démo — Le front StockLine sur S3 privé derrière CloudFront

L'architecture de la démo



Démo 7-1 — Front S3 privé + CloudFront + OAC

Fichier : `demos/06-cl-aws1/demo-7-1-cloudfront-acm.md` — 50 min

1. bucket **privé** (Block Public Access ON), preuve par **403** ;
2. création de l'**OAC**, puis de la **distribution** (JSON commenté ligne à ligne) ;
3. la **bucket policy** qui n'autorise QUE cette distribution (grille du J4) ;
4. **x-cache**: **Miss** puis **Hit** — le cache au travail ; le front en **HTTPS** dans le navigateur ;
5. mise à jour du front → rien ne change (cache !) → **invalidation** → v2 ;
6. et pour finir, **sans rien créer** : le scénario « vrai domaine » complet (Route 53 + ACM + alias).

Pas de domaine à acheter : le domaine `*.cloudfront.net` fait tout le travail pédagogique.

Avec un vrai domaine : la checklist complète

Ce qui changerait avec `stockline.fr` (à connaître pour l'entretien comme pour le TP2 en soutenance) :

1. **Acheter/transférer** le domaine (~12 /an) → *zone hébergée Route53* (0,50/mois) ;
2. **ACM us-east-1** : certificat `stockline.fr` + `*.stockline.fr`, **validation DNS** (le CNAME dans la zone) ;
3. **Distribution** : ajouter l'*alternate domain name* `www.stockline.fr` + attacher le certificat ;
4. **Route 53** : enregistrement **A alias** `www` → la distribution (et l'apex → alias aussi) ;
5. (API en HTTPS : 2^e certificat **eu-west-3** sur le listener 443 de l'ALB.)

Coût total : ~18 \$/an. Temps : ~30 min, dont 25 à attendre les propagations. Le HTTPS n'est plus jamais une excuse.

Récap — les points clés du jour

- **Route 53** : zones publiques/privées ; politiques **simple** / **pondéré (canary)** / **latence (multi-régions)** / **failover (secours)** — imbriquables ; un failover sans health check n'existe pas.
- **Alias vs CNAME** : cible AWS → alias (gratuit, apex OK, evaluate target health) ; cible externe → CNAME, jamais à l'apex.
- **CloudFront** : cache **par edge et par TTL** ; **CachingOptimized** pour le statique, **CachingDisabled** pour **/api/*** ; invalidation = urgence, **versionnement des noms** = déploiement.
- **OAC** : bucket privé + policy « Principal CloudFront + SourceArn » — plus jamais de bucket public.
- **ACM** : gratuit, renouvellement auto (validation DNS) ; **us-east-1 pour CloudFront**, région locale pour l'ALB.
- 💰 CloudFront quasi gratuit à notre échelle — l'inverse de l'ALB.

Teardown du soir — non négociable

```
# 1. Désactiver PUIS supprimer la distribution (deux temps obligatoires) :
ETAG=$(aws cloudfront get-distribution-config --id $DIST_ID --query ETag --output
text)
aws cloudfront get-distribution-config --id $DIST_ID --query DistributionConfig \
  | sed 's/"Enabled": true/"Enabled": false/' > /tmp/dist-off.json
aws cloudfront update-distribution --id $DIST_ID \
  --distribution-config file:///tmp/dist-off.json --if-match $ETAG
aws cloudfront wait distribution-deployed --id $DIST_ID
ETAG=$(aws cloudfront get-distribution-config --id $DIST_ID --query ETag --output
text)
aws cloudfront delete-distribution --id $DIST_ID --if-match $ETAG

# 2. OAC + bucket :
aws cloudfront delete-origin-access-control --id $OAC_ID --if-match \
  $(aws cloudfront get-origin-access-control --id $OAC_ID --query ETag --output
text)
aws s3 rb s3://${PREFIX}-stockline-front --force

# 3. Preuve :
aws cloudfront list-distributions --query 'DistributionList.Items[].Id'
```

Quiz du jour (1/5)

Question 1 — Pourquoi est-il impossible de créer un enregistrement CNAME sur **stockline.fr** (l'apex de la zone), et quelle est la solution Route 53 pour faire pointer l'apex vers un ALB ?

Question 2 — Le marketing veut tester une nouvelle version du site sur 10 % du trafic, avec possibilité de revenir en arrière en une minute. Quelle politique de routage Route 53 proposez-vous et comment fonctionne le retour arrière ?

Quiz du jour (2/5)

Question 3 — Quelle est la différence entre la politique de routage « latence » et la politique « géolocalisation » ? Donnez un cas d'usage où seule la géolocalisation convient.

Question 4 — Un enregistrement failover est configuré avec un primaire et un secondaire, mais sans health check associé au primaire. Que se passe-t-il quand le site principal tombe ?

Quiz du jour (3/5)

Question 5 — Vous poussez une nouvelle version de `index.html` dans le bucket d'origine. Un utilisateur parisien qui visitait le site hier voit l'ancienne version, un utilisateur de Sydney qui n'est jamais venu voit la nouvelle. Expliquez.

Question 6 — Citez les deux problèmes de la pratique « invalider `/*` à chaque déploiement » et l'alternative recommandée pour les fichiers statiques.

Quiz du jour (4/5)

Question 7 — Qu'est-ce qu'un Origin Access Control, et quels sont les trois éléments de la bucket policy qui garantissent que SEULE votre distribution peut lire le bucket ?

Question 8 — Vous demandez un certificat ACM dans eu-west-3 pour l'attacher à votre distribution CloudFront : le certificat n'apparaît pas dans la console CloudFront. Pourquoi, et où faut-il le demander ? Et pour le listener HTTPS de votre ALB parisien ?

Quiz du jour (5/5)

Question 9 — Pourquoi préfère-t-on la validation DNS à la validation e-mail pour un certificat ACM ? Que se passe-t-il au moment du renouvellement dans chaque cas ?

Question 10 — Dans l'architecture cible du TP2, le comportement `/api/*` de la distribution pointe vers l'ALB avec la politique `CachingDisabled`. Justifiez ce choix : que se passerait-il avec `CachingOptimized` sur `GET /stocks/1` ?

Réponses détaillées : guide formateur du bloc (jours 5-8).

À demain !

Jour 8 — Exploiter et payer

Votre architecture sait encaisser les pannes et parler HTTPS. Mais qui vous **prévient** quand ça va mal ? Qui surveille la **facture** ? Et qui refait tout ça à la main chaque matin ? Demain : **CloudWatch**, **SNS**, **CloudTrail**, **Cost Explorer**, **boto3** — et la synthèse des 8 jours avant le TP2.

🔪 Avant de partir : distribution supprimée, bucket front détruit, compte propre.